KEMENTERIAN PENDIDIKAN, KEBUDAYAAN, RISET, DAN TEKNOLOGI
POLITEKNIK NEGERI MALANG
JURUSAN TEKNOLOGI INFORMASI
Jl. Soekarno Hatta No. 9, Jatimulyo, Lowokwaru, Malang 65141
Telp. (0341) 404424 – 404425, Fax (0341) 404420
https://jti.polinema.ac.id

Mata Kuliah	:	Pemrograman Mobile
Program Studi	:	D4 – Teknik Informatika
Semester	:	5

Kelas	:	TI-3C
NIM	:	244107020086
Nama	:	Zacky Rio Orlando
Jobsheet Ke-	:	5

Laporan Jobsheet

Praktikum 1 — Preferensi dengan SharedPreferences

Langkah	Jawaban/Deskripsi
1	**Langkah 1. Buat repository preferensi lib/data/prefs.dart**
2	**Langkah 2. Buat provider preferensi lib/providers/prefs_providers.dart**

week5_offline_notes > lib > data > prefs.dart > ...

```dart
import 'package:shared_preferences/shared_preferences.dart';

class PrefsRepository {
  static const _darkModeKey = 'dark_mode';
  static const _lastOpenedKey = 'last_opened_at';
  Future<bool> getDarkMode() async {
    final prefs = await SharedPreferences.getInstance();
    return prefs.getBool(_darkModeKey) ?? false;
  }

  Future<void> setDarkMode(bool value) async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setBool(_darkModeKey, value);
  }

  Future<void> markOpenedNow() async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setString(_lastOpenedKey, DateTime.now().toIso8601String());
  }

  Future<String?> getLastOpened() async {
    final prefs = await SharedPreferences.getInstance();
    return prefs.getString(_lastOpenedKey);
  }
}

```

week5_offline_notes > lib > providers > prefs_providers.dart > DarkModeNotifier > toggle

```dart
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../data/prefs.dart';

final prefsRepositoryProvider = Provider<PrefsRepository>(
  (ref) => PrefsRepository(),
); // Provider
final darkModeProvider = AsyncNotifierProvider<DarkModeNotifier, bool>(
  DarkModeNotifier.new,
);

class DarkModeNotifier extends AsyncNotifier<bool> {
  @override
  Future<bool> build() => ref.watch(prefsRepositoryProvider).getDarkMode();
  Future<void> toggle() async {
    final previous = state.value ?? false;
    final next = !previous;
    // Optimistic update: UI langsung berubah tanpa menunggu disk.
    state = AsyncData(next);
    try {
      await ref.read(prefsRepositoryProvider).setDarkMode(next);
    } catch (_) {
      // Gagal menyimpan: kembalikan ke nilai sebelumnya.
      state = AsyncData(previous);
      rethrow;
    }
  }
}

/// Membaca waktu terakhir dibuka (dari sesi sebelumnya),
/// lalu mencatat waktu sekarang untuk sesi berikutnya.
final lastOpenedProvider = FutureProvider<String?>((ref) async {
  final repo = ref.watch(prefsRepositoryProvider);
  final previous = await repo.getLastOpened();
  await repo.markOpenedNow();
  return previous;
});

```

3	**Langkah 3. Buat halaman pengaturan lib/pages/settings_page.dart.**

week5_offline_notes > lib > pages > settings_page.dart > ...

```dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../providers/prefs_providers.dart';

class SettingsPage extends ConsumerWidget {
  const SettingsPage({super.key});
  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final darkAsync = ref.watch(darkModeProvider);
    final lastOpenedAsync = ref.watch(lastOpenedProvider);
    return Scaffold(
      appBar: AppBar(title: const Text('Pengaturan')),
      body: ListView(
        children: [
          darkAsync.when(
            loading: () => const ListTile(
              title: Text('Memuat preferensi...'),
              trailing: SizedBox(
                width: 24,
                height: 24,
                child: CircularProgressIndicator(strokeWidth: 2),
              ), // SizedBox
            ), // ListTile
            error: (e, _) => ListTile(
              leading: const Icon(Icons.error_outline),
              title: const Text('Gagal memuat preferensi'),
              subtitle: Text('$e'),
            ), // ListTile
            data: (isDark) => SwitchListTile(
              secondary: const Icon(Icons.dark_mode_outlined),
              title: const Text('Tema gelap'),
              subtitle: const Text('Disimpan di SharedPreferences'),
              value: isDark,
              onChanged: (_) => ref.read(darkModeProvider.notifier).toggle(),
            ), // SwitchListTile
          ),
          ListTile(
            leading: const Icon(Icons.history),
            title: const Text('Terakhir dibuka'),
            subtitle: Text(
              lastOpenedAsync.when(
                data: (value) => value ?? 'Baru pertama kali dibuka',
                loading: () => 'Memuat...',
                error: (e, _) => 'Error: $e',
              ),
            ), // Text
          ), // ListTile
        ],
      ), // ListView
    ); // Scaffold
  }
}
```

4	**Langkah 4. Ganti isi lib/main.dart agar tema mengikuti preferensi.**

week5_offline_notes > lib > main.dart > ...

```dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import 'pages/settings_page.dart';
import 'providers/prefs_providers.dart';

void main() {
  WidgetsFlutterBinding.ensureInitialized();
  runApp(const ProviderScope(child: OfflineNotesApp()));
}

class OfflineNotesApp extends ConsumerWidget {
  const OfflineNotesApp({super.key});
  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final isDark = ref.watch(darkModeProvider).value ?? false;
    return MaterialApp(
      title: 'Offline Notes',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorSchemeSeed: Colors.indigo,
        brightness: Brightness.light,
      ), // ThemeData
      darkTheme: ThemeData(
        colorSchemeSeed: Colors.indigo,
        brightness: Brightness.dark,
      ), // ThemeData
      themeMode: isDark ? ThemeMode.dark : ThemeMode.light,
      home: const SettingsPage(), // diganti NotesPage pada Praktikum 3
    ); // MaterialApp
  }
}

```

5

Langkah 5. Jalankan dan uji persistensi

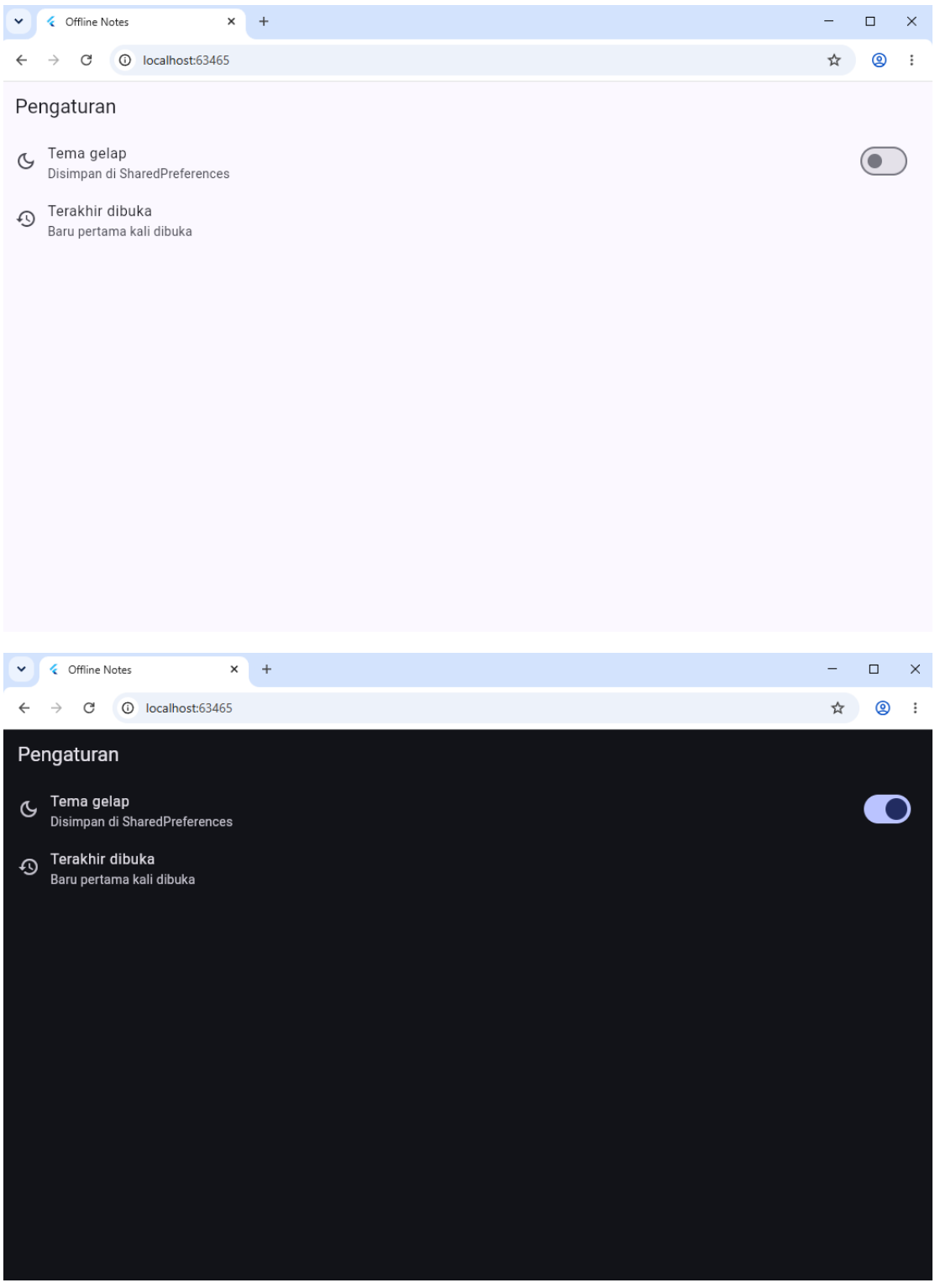
```
PS C:\Users\Ryoo\my_first_app\week5_offline_notes> flutter run
Connected devices:
Windows (desktop) • windows • windows-x64    • Microsoft Windows [Version 10.0.19045.6466]
Chrome (web)      • chrome  • web-javascript • Google Chrome 154.0.8037.93
Edge (web)        • edge    • web-javascript • Microsoft Edge 154.0.4258.48
[1]: Windows (windows)
[2]: Chrome (chrome)
[3]: Edge (edge)
Please choose one (or "q" to quit): 2
Launching lib\main.dart on Chrome in debug mode...
Waiting for connection from debug service on Chrome...
```

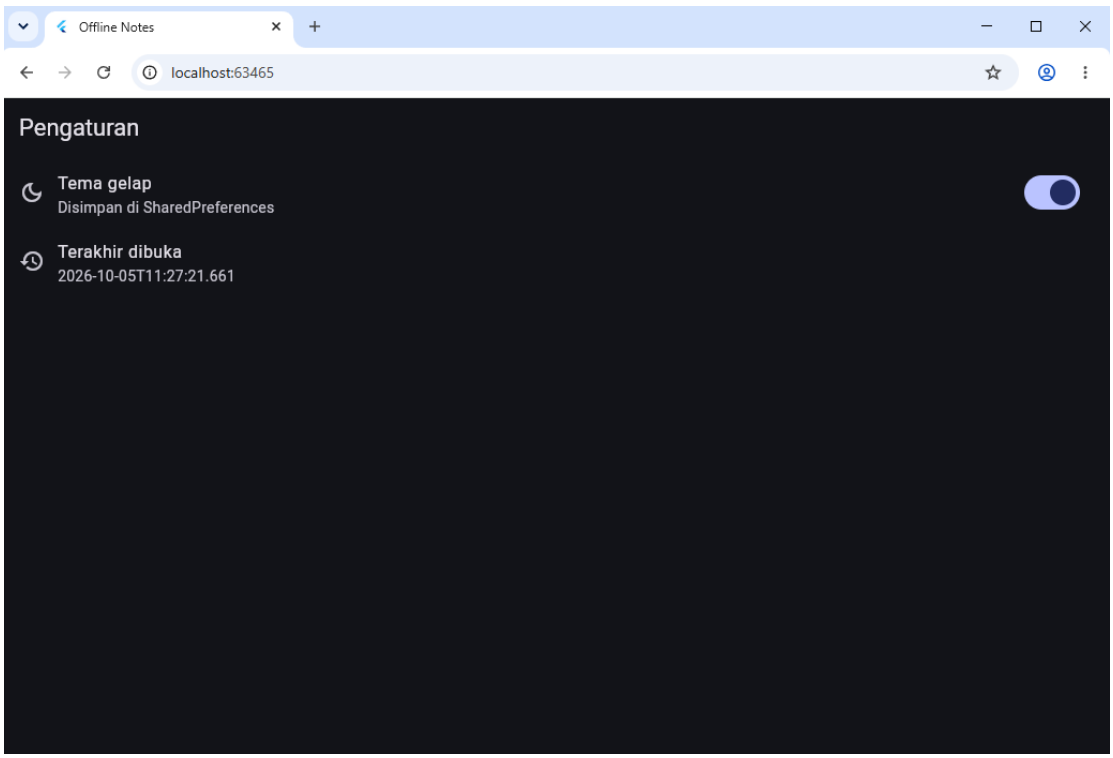
1. Aktifkan switch **Tema gelap**. Tema harus langsung berubah.

2. Tutup aplikasi sepenuhnya (geser dari daftar aplikasi terbaru), lalu buka kembali. Tema gelap harus **tetap aktif**.

3. Perhatikan baris **Terakhir dibuka**: pada pembukaan pertama tertulis “Baru pertama kali dibuka”, pada pembukaan berikutnya berisi waktu sesi sebelumnya.

4. Ambil screenshot tema terang dan tema gelap, simpan ke screenshots/p1-tematerang.png dan screenshots/p1-tema-gelap.png.


Offline Notes
localhost:63465
Pengaturan
Tema gelap
Disimpan di SharedPreferences
Terakhir dibuka
Baru pertama kali dibuka
Offline Notes
localhost:63465
Pengaturan
Tema gelap
Disimpan di SharedPreferences
Terakhir dibuka
Baru pertama kali dibuka



Pada saat di run terlihat bahwa awalnya temanya terang kemudian saat switchnya diaktifkan langsung berubah menjadi gelap dan saat aplikasinya di tutup (reload website) dan dibuka kembali terjadi perubahan pada Terakhir dibuka yang awalnya "Baru pertama kali dibuka" menjadi format tanggal-bulan-tahun beserta waktunya.

Pertanyaan Praktikum 1

1. Mengapa SharedPreferences.getInstance() tidak boleh dipanggil di dalam methodbuild() widget?

   Jawaban:

   - Karena build() dapat dijalankan berkali-kali setiap kali terjadi perubahan pada UI. Jika dipanggil di sana, proses mengambil data dari penyimpanan bisa dilakukan berulang-ulang dan membuat aplikasi menjadi tidak efisien.

2. Jelaskan alur data dari saat switch ditekan sampai nilai tersimpan di disk dan tema berubah.

   Jawaban:

   - Saat switch tema ditekan, fungsi toggle() pada DarkModeNotifier akan dijalankan. Fungsi tersebut mengambil nilai tema saat ini, kemudian membaliknya menjadi nilai baru. Nilai baru

langsung diberikan ke state sehingga tampilan tema aplikasi segera berubah menjadi terang atau gelap. Setelah itu, nilai tersebut disimpan ke SharedPreferences melalui PrefsRepository.

3. Apa kelebihan dan risiko pendekatan *optimistic update* pada toggle()?
   Jawaban:
   - Kelebihan *optimistic update* adalah saat switch ditekan, tampilan tema langsung berubah sehingga aplikasi terasa lebih cepat dan responsif. Kekurangannya, jika penyimpanan data gagal, nilai yang tampil bisa tidak tersimpan dengan benar.

Praktikum 2 — Model dan Database SQLite

Langkah	Jawaban/Deskripsi
1	**Langkah 1. Buat model catatan lib/data/local/note.dart.**

week5_offline_notes > lib > data > local > note.dart > Note > Note.fromMap

```dart
class Note {
  const Note({
    this.id,
    required this.title,
    this.body = '',
    required this.updatedAt,
    this.dirty = false,
  });
  final int? id;
  final String title;
  final String body;
  final DateTime updatedAt;
  final bool dirty;
  Note copyWith({
    int? id,
    String? title,
    String? body,
    DateTime? updatedAt,
    bool? dirty,
  }) {
    return Note(
      id: id ?? this.id,
      title: title ?? this.title,
      body: body ?? this.body,
      updatedAt: updatedAt ?? this.updatedAt,
      dirty: dirty ?? this.dirty,
    );
  }

  Map<String, Object?> toMap() => {
    'id': id,
    'title': title,
    'body': body,
    'updated_at': updatedAt.toIso8601String(),
    'dirty': dirty ? 1 : 0,
  };
  factory Note.fromMap(Map<String, Object?> map) {
    return Note(
      id: (map['id'] as num?)?.toInt(),
      title: map['title'] as String? ?? '',
      body: map['body'] as String? ?? '',
      updatedAt:
          DateTime.tryParse(map['updated_at'] as String? ?? '') ??
          DateTime.fromMillisecondsSinceEpoch(0),
      dirty: ((map['dirty'] as num?)?.toInt() ?? 0) == 1,
    ); // Note
  }
}
```

2	**Langkah 2. Buat pembuka database lib/data/local/db.dart**

week5_offline_notes > lib > data > local > db.dart > ...

```dart
import 'package:path/path.dart' as p;
import 'package:sqflite/sqflite.dart';

Future<Database> openNotesDb() async {
  final dir = await getDatabasesPath();
  return openDatabase(
    p.join(dir, 'offline_notes.db'),
    version: 1,
    onCreate: (db, version) async {
      await db.execute('''
CREATE TABLE notes(
id INTEGER PRIMARY KEY AUTOINCREMENT,
title TEXT NOT NULL,
body TEXT NOT NULL DEFAULT '',
updated_at TEXT NOT NULL,
dirty INTEGER NOT NULL DEFAULT 0
)
''');
      await db.execute('''
CREATE TABLE cached_posts(
id INTEGER PRIMARY KEY,
payload TEXT NOT NULL,
cached_at TEXT NOT NULL
)
''');
    },
  );
}

```

3	**Langkah 3. Buat repository lib/data/repositories/note_repository.dart.**

week5_offline_notes > lib > data > repositories > note_repository.dart > ...

```dart
import 'package:sqflite/sqflite.dart';

import '../local/db.dart';
import '../local/note.dart';

class NoteRepository {
  NoteRepository({Future<Database> Function()? openDb})
    : _openDb = openDb ?? openNotesDb;
  final Future<Database> Function() _openDb;
  Future<List<Note>> fetchNotes() async {
    final db = await _openDb();
    final rows = await db.query('notes', orderBy: 'updated_at DESC');
    return rows.map(Note.fromMap).toList();
  }

  Future<Note?> getNoteById(int id) async {
    final db = await _openDb();
    final rows = await db.query(
      'notes',
      where: 'id = ?',
      whereArgs: [id],
      limit: 1,
    );
    return rows.isEmpty ? null : Note.fromMap(rows.first);
  }

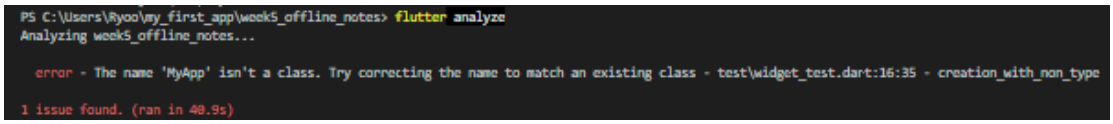
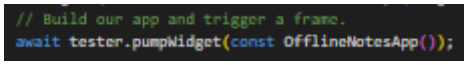
  Future<Note> addNote({required String title, String body = ''}) async {
    final db = await _openDb();
    final note = Note(
      title: title,
      body: body,
      updatedAt: DateTime.now(),
      dirty: true,
    ); // Note
    final id = await db.insert('notes', note.toMap());
    return note.copyWith(id: id);
  }

  Future<void> updateNote(Note note) async {
    if (note.id == null) {
      throw ArgumentError('Catatan belum memiliki id');
    }
    final db = await _openDb();
    final updated = note.copyWith(updatedAt: DateTime.now(), dirty: true);
    await db.update(
      'notes',
      updated.toMap(),
      where: 'id = ?',
      whereArgs: [note.id],
    );
  }

  Future<void> deleteNote(int id) async {
    final db = await _openDb();
    await db.delete('notes', where: 'id = ?', whereArgs: [id]);
  }

  Future<int> countDirty() async {
    final db = await _openDb();
    final rows = await db.rawQuery(
      'SELECT COUNT(*) AS c FROM notes WHERE dirty = 1',
    );
    return (rows.first['c'] as num?)?.toInt() ?? 0;
  }

  Future<void> markAllSynced() async {
    final db = await _openDb();
    await db.update('notes', {'dirty': 0}, where: 'dirty = 1');
  }
}
```

4	Checkpoint Praktikum 2 • *flutter analyze* tidak menampilkan error pada folder *lib/data/*. • Anda bisa menjelaskan fungsi setiap kolom tabel *notes*. • Commit: *feat(db): model Note, skema SQLite, dan NoteRepository*.  Tidak ada error pada folder lib/data, error tersebut muncul karena file test/widget_test.dart masih menggunakan class MyApp, sedangkan pada main.dart class aplikasi sudah menggunakan OfflineNotesApp. Diubah menjadi seperti ini  Penjelasan fungsi setiap kolom table notes - Id: ID unik untuk setiap catatan - Title: Menyimpan judul catatan - Body: Menyimpan isi catatan - Updated_at: Menyimpan waktu terakhir catatan diperbarui - Dirty: Menandai apakah catatan belum disinkronkan ke server

Pertanyaan Praktikum 2

1. Mengapa kolom dirty bertipe INTEGER dan bukan BOOLEAN?

   Jawaban:

   - Karena SQLite tidak memiliki tipe data BOOLEAN secara khusus. Oleh karena itu, nilai dirty disimpan sebagai angka, yaitu 0 untuk false dan 1 untuk true.

2. Apa fungsi parameter openDb pada constructor NoteRepository?

   Jawaban:

   - Digunakan untuk menentukan fungsi yang membuka database. Dengan adanya parameter ini, repository lebih mudah diuji karena kita bisa memberikan database lain tanpa harus selalu menggunakan database asli.

3. Mengapa query memakai where: 'id = ?' dan whereArgs, bukan interpolasi string?

   Jawaban:

- Karena penggunaan ? dan whereArgs lebih aman dan dapat mencegah masalah seperti SQL injection.

4. Apa yang terjadi jika Anda menambah kolom baru di onCreate tanpa menaikkan version?
   Jawaban:

   - Database yang sudah pernah dibuat tidak akan menjalankan `onCreate` lagi, sehingga kolom baru tersebut tidak akan ditambahkan ke database lama. Akibatnya, aplikasi bisa mengalami error ketika mencoba menggunakan kolom yang belum ada.

Praktikum 3 — Halaman Catatan Offline dengan Riverpod

Langkah	Jawaban/Deskripsi
1	**Langkah 1. Buat provider catatan lib/providers/note_providers.dart.**

week5_offline_notes > lib > providers > note_providers.dart

```dart
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../data/local/note.dart';
import '../data/repositories/note_repository.dart';

final noteRepositoryProvider = Provider<NoteRepository>(
  (ref) => NoteRepository(),
); // Provider
final notesProvider = FutureProvider<List<Note>>(
  (ref) => ref.watch(noteRepositoryProvider).fetchNotes(),
);
final dirtyCountProvider = FutureProvider<int>(
  (ref) => ref.watch(noteRepositoryProvider).countDirty(),
);
final noteActionsProvider = Provider<NoteActions>((ref) => NoteActions(ref));

/// Kumpulan aksi yang mengubah data. Setiap mutasi diakhiri invalidate
/// agar provider baca (notes & dirty count) memuat ulang dari database.
class NoteActions {
  NoteActions(this._ref);
  final Ref _ref;
  NoteRepository get _repo => _ref.read(noteRepositoryProvider);
  Future<void> add(String title, String body) async {
    await _repo.addNote(title: title, body: body);
    _refresh();
  }

  Future<void> update(Note note) async {
    await _repo.updateNote(note);
    _refresh();
  }

  Future<void> delete(int id) async {
    await _repo.deleteNote(id);
    _refresh();
  }

  void _refresh() {
    _ref.invalidate(notesProvider);
    _ref.invalidate(dirtyCountProvider);
  }
}

```

2	**Langkah 2. Buat dialog form catatan lib/widgets/note_form_dialog.dart.**

week5_offline_notes > lib > widgets > note_form_dialog.dart > ...

```dart
import 'package:flutter/material.dart';

import '../data/local/note.dart';

typedef NoteFormResult = ({String title, String body});

class NoteFormDialog extends StatefulWidget {
  const NoteFormDialog({super.key, this.initial});
  final Note? initial;
  @override
  State<NoteFormDialog> createState() => _NoteFormDialogState();
}

class _NoteFormDialogState extends State<NoteFormDialog> {
  late final TextEditingController _title = TextEditingController(
    text: widget.initial?.title ?? '',
  );
  late final TextEditingController _body = TextEditingController(
    text: widget.initial?.body ?? '',
  );
  String? _error;
  @override
  void dispose() {
    _title.dispose();
    _body.dispose();
    super.dispose();
  }

  void _submit() {
    final title = _title.text.trim();
    if (title.isEmpty) {
      setState(() => _error = 'Judul wajib diisi');
      return;
    }
    Navigator.of(context)
        .pop<NoteFormResult>((title: title, body: _body.text.trim()));
  }

  @override
  Widget build(BuildContext context) {
    return AlertDialog(
      title: Text(widget.initial == null ? 'Catatan baru' : 'Ubah catatan'),
      content: Column(
        mainAxisSize: MainAxisSize.min,
        children: [
          TextField(
            controller: _title,
            autofocus: true,
            decoration: InputDecoration(labelText: 'Judul', errorText: _error),
          ), // TextField
          const SizedBox(height: 8),
          TextField(
            controller: _body,
            maxLines: 4,
            decoration: const InputDecoration(labelText: 'Isi catatan'),
          ), // TextField
        ],
      ), // Column
      actions: [
        TextButton(
          onPressed: () => Navigator.of(context).pop(),
          child: const Text('Batal'),
        ), // TextButton
        FilledButton(onPressed: _submit, child: const Text('Simpan')),
      ],
    ); // AlertDialog
  }
}
```

3	**Langkah 3. Buat halaman catatan lib/pages/notes_page.dart.**

week5_offline_notes > lib > pages > note_page.dart > ...

```dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../data/local/note.dart';
import '../providers/note_providers.dart';
import '../widgets/note_form_dialog.dart';
import 'settings_page.dart';

class NotesPage extends ConsumerWidget {
  const NotesPage({super.key});
  Future<void> _openForm(
    BuildContext context,
    WidgetRef ref, [
    Note? note,
  ]) async {
    final result = await showDialog<NoteFormResult>(
      context: context,
      builder: (_) => NoteFormDialog(initial: note),
    );
    if (result == null) return; // dibatalkan
    final actions = ref.read(noteActionsProvider);
    if (note == null) {
      await actions.add(result.title, result.body);
    } else {
      await actions.update(
        note.copyWith(title: result.title, body: result.body),
      );
    }
  }

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final notesAsync = ref.watch(notesProvider);
    final dirty = ref.watch(dirtyCountProvider).value ?? 0;
    return Scaffold(
      appBar: AppBar(
        title: const Text('Offline Notes'),
        actions: [
          Tooltip(
            message: 'Catatan belum tersinkron',
            child: Badge(
              isLabelVisible: dirty > 0,
              label: Text('$dirty'),
              child: const Icon(Icons.cloud_upload_outlined),
            ), // Badge
          ), // Tooltip
          IconButton(
```

```
            tooltip: 'Pengaturan',
            icon: const Icon(Icons.settings_outlined),
            onPressed: () => Navigator.of(context)
                .push(MaterialPageRoute(builder: (_) => const SettingsPage())),
          ), // IconButton
        ],
      ), // AppBar
      body: notesAsync.when(
        loading: () => const Center(child: CircularProgressIndicator()),
        error: (e, _) => _ErrorView(
          message: 'Gagal membaca database: $e',
          onRetry: () => ref.invalidate(notesProvider),
        ), // _ErrorView
        data: (notes) {
          if (notes.isEmpty) return const _EmptyView();
          return ListView.separated(
            itemCount: notes.length,
            separatorBuilder: (context, index) => const Divider(height: 1),
            itemBuilder: (context, index) {
              final note = notes[index];
              return ListTile(
                leading: Icon(
                  note.dirty ? Icons.cloud_off : Icons.cloud_done,
                  color: note.dirty ? Colors.orange : Colors.green,
                ), // Icon
                title: Text(note.title),
                subtitle: Text(
                  note.body.isEmpty ? '(tanpa isi)' : note.body,
                  maxLines: 1,
                  overflow: TextOverflow.ellipsis,
                ), // Text
                onTap: () => _openForm(context, ref, note),
                trailing: IconButton(
                  tooltip: 'Hapus',
                  icon: const Icon(Icons.delete_outline),
                  onPressed: () =>
                      ref.read(noteActionsProvider).delete(note.id!),
                ), // IconButton
              ); // ListTile
            },
          ); // ListView.separated
        },
      ),
      floatingActionButton: FloatingActionButton.extended(
```

```
            onPressed: () => _openForm(context, ref),
            icon: const Icon(Icons.add),
            label: const Text('Catatan'),
          ), // FloatingActionButton.extended
        ); // Scaffold
      }
    }

    class _EmptyView extends StatelessWidget {
      const _EmptyView();
      @override
      Widget build(BuildContext context) {
        return const Center(
          child: Column(
            mainAxisSize: MainAxisSize.min,
            children: [
              Icon(Icons.note_alt_outlined, size: 64),
              SizedBox(height: 12),
              Text('Belum ada catatan. Tekan + untuk menambah.'),
            ],
          ), // Column
        ); // Center
      }
    }

    class _ErrorView extends StatelessWidget {
      const _ErrorView({required this.message, required this.onRetry});
      final String message;
      final VoidCallback onRetry;
      @override
      Widget build(BuildContext context) {
        return Center(
          child: Padding(
            padding: const EdgeInsets.all(24),
            child: Column(
              mainAxisSize: MainAxisSize.min,
              children: [
                const Icon(Icons.error_outline, size: 48),
                const SizedBox(height: 12),
                Text(message, textAlign: TextAlign.center),
                const SizedBox(height: 12),
                FilledButton(onPressed: onRetry, child: const Text('Coba lagi')),
              ],
            ), // Column
```

```
          ), // Padding
        ); // Center
      }
    }
```

4 **Langkah 4. Jadikan NotesPage sebagai halaman utama di lib/main.dart**

```
import 'pages/notes_page.dart';    Duplicat
import 'providers/prefs_providers.dart';
```

```
      themeMode: isDark ? ThemeMode.dark : ThemeMode.light,
      home: const NotesPage(), // diganti NotesPage pada Praktikum 3
    ); // MaterialApp
```

5 **Langkah 5. Uji CRUD dan mode pesawat.**

No	Skenario	Langkah	Hasil yang diharapkan
1	Empty state	Jalankan aplikasi pertama kali	Ikon + teks “Belum ada catatan”
2	Create	Tekan + **Catatan**, isi judul, Simpan	Catatan muncul paling atas, ikon awan oranye, badge = 1

No	Skenario	Langkah	Hasil yang diharapkan
3	Validasi	Simpan dengan judul kosong	Muncul pesan "Judul wajib diisi", dialog tidak tertutup
4	Update	Ketuk catatan, ubah isi, Simpan	Isi berubah dan catatan naik ke posisi teratas
5	Delete	Tekan ikon tempat sampah	Catatan hilang, badge berkurang bila catatan itu dirty
6	Persistensi	Tutup aplikasi lalu buka kembali	Seluruh catatan masih ada
7	Mode pesawat	Aktifkan mode pesawat, ulangi skenario 2, 4, 5	Semua tetap berfungsi tanpa error

1. Empty State


3:18
Offline Notes
Belum ada catatan. Tekan + untuk menambah.
Catatan

2. Create


3:21
Offline Notes
Praktikum 3
Ini adalah uji coba dari create dari pr...
Catatan

3. Validasi


3:25
Offline Notes
Testing
test 1 2 3 test
Praktikum 3
Ini adalah uji coba dari create dari pr...
Catatan baru
Judul
Judul wajib diisi
Isi catatan
ini uji coba validasi dengan judul kosong
Batal
Simpan
Catatan

4. Update


3:26
Offline Notes
Praktikum 3 - UPDATE
Ini adalah uji coba dari update dari pr...
Testing
test 1 2 3 test
Catatan

5. Delete

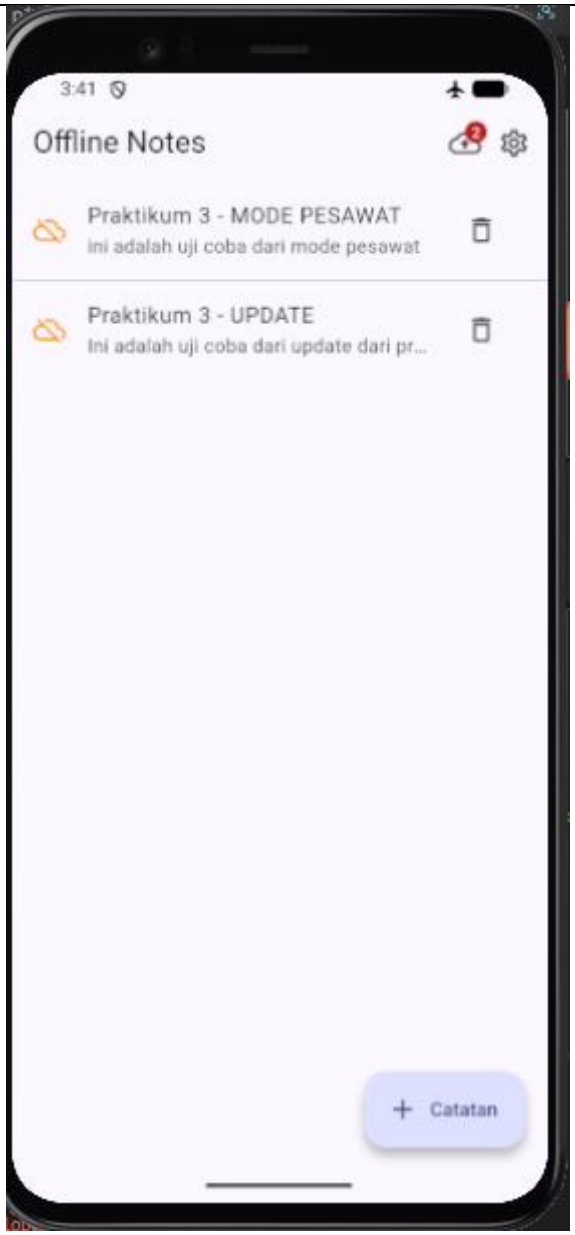


6. Persistensi

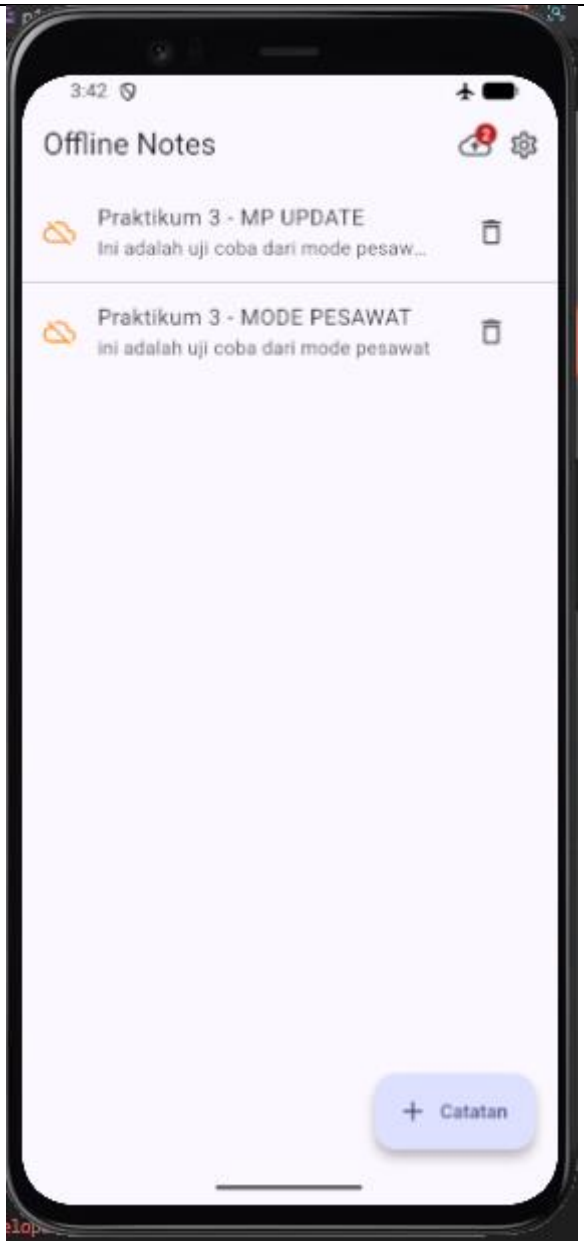


7. Mode Pesawat

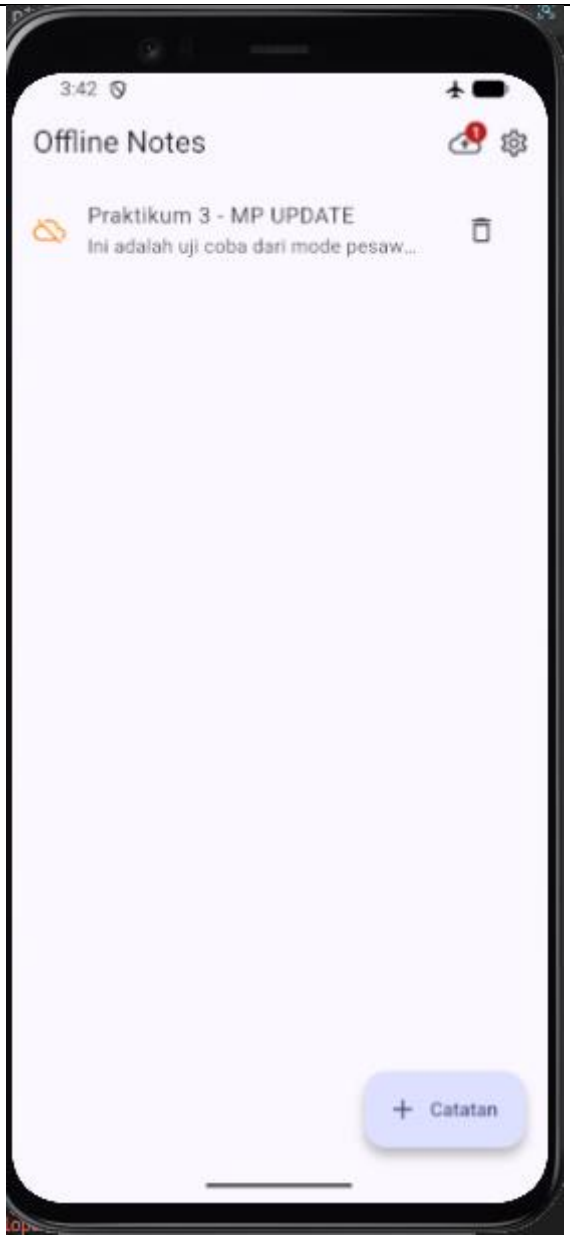
1. Create


3:41
Offline Notes
Praktikum 3 - MODE PESAWAT
ini adalah uji coba dari mode pesawat
Praktikum 3 - UPDATE
Ini adalah uji coba dari update dari pr...
Catatan

2. Update


3:42
Offline Notes
Praktikum 3 - MP UPDATE
Ini adalah uji coba dari mode pesaw...
Praktikum 3 - MODE PESAWAT
ini adalah uji coba dari mode pesawat
Catatan

3. Delete



Berdasarkan hasil pengujian, semua skenario berjalan dengan normal. Fitur tambah, edit, hapus, validasi, dan penyimpanan catatan tetap berfungsi dengan baik, termasuk saat mode pesawat diaktifkan. Jadi, aplikasi sudah berhasil menerapkan penyimpanan lokal dan dapat digunakan secara offline tanpa error.

Pertanyaan Praktikum 3

1. Mengapa setelah setiap mutasi perlu meng-invalidate notesProvider **dan**dirtyCountProvider? Apa yang terjadi jika hanya salah satu?

   Jawaban:

- Karena keduanya menampilkan data yang berbeda. notesProvider untuk memperbarui daftar catatan, sedangkan dirtyCountProvider untuk memperbarui jumlah catatan yang belum tersinkron. Jika hanya salah satu yang di-invalidate, bagian lainnya bisa menampilkan data lama.

2. Bagaimana cara Anda memicu state **error** secara sengaja untuk menguji tampilan_ErrorView?

   Jawaban:

   - Dengan mengubah atau membuat kode pada proses pengambilan data agar menghasilkan error, misalnya koneksi database gagal. Setelah error terjadi, _ErrorView akan muncul dan menampilkan pesan kesalahan.

3. Mengapa aplikasi tetap berfungsi dalam mode pesawat walaupun tidak ada kode khusus untuk mode offline?

   Jawaban:

   - Aplikasi tetap berjalan saat mode pesawat karena data catatan disimpan di SQLite pada perangkat. Jadi, fitur tambah, ubah, dan hapus tidak bergantung pada internet.

Praktikum 4 — Offline-First: Cache-First dan Sinkronisasi

Langkah	Jawaban/Deskripsi
1	**Langkah 1. Buat saklar offline lib/providers/offline_providers.dart.**

week5_offline_notes > lib > providers > offline_providers.dart > ...

```dart
import 'package:flutter_riverpod/flutter_riverpod.dart';

/// Simulasi offline yang deterministik untuk demo dan testing,
/// sehingga tidak bergantung pada kondisi Wi-Fi kelas.
final forceOfflineProvider = NotifierProvider<ForceOfflineNotifier, bool>(
  ForceOfflineNotifier.new,
);

class ForceOfflineNotifier extends Notifier<bool> {
  @override
  bool build() => false;
  void toggle() => state = !state;
}
```

2	**Langkah 2. Buat logika sinkronisasi lib/data/sync.dart.**
	week5_offline_notes > lib > data > sync.dart > ...
	`import 'local/note.dart';`
	`import 'repositories/note_repository.dart';`
	
	`class OfflineException implements Exception {`
	`  const OfflineException(this.message);`
	`  final String message;`
	`  @override`
	`  String toString() => 'OfflineException: $message';`
	`}`
	
	`/// Mengirim catatan dirty ke "server" (simulasi) lalu menandainya bersih.`
	`/// Mengembalikan jumlah catatan yang tersinkron.`
	`Future<int> syncNotes(`
	`  NoteRepository repo, {`
	`  bool offline = false,`
	`  Duration latency = const Duration(seconds: 1),`
	`}) async {`
	`  if (offline) {`
	`  throw const OfflineException('Perangkat offline, sinkronisasi ditunda.');`
	`  }`
	`  final dirtyCount = await repo.countDirty();`
	`  if (dirtyCount == 0) return 0;`
	`  // Simulasi upload. Pada project nyata: kirim tiap catatan dirty`
	`  // ke REST API, lalu tandai bersih HANYA bila server menjawab 2xx.`
	`  await Future.delayed(latency);`
	`  await repo.markAllSynced();`
	`  return dirtyCount;`
	`}`
	
	`/// Aturan konflik: last-write-wins berdasarkan updatedAt.`
	`Note resolveConflict(Note local, Note remote) {`
	`  return remote.updatedAt.isAfter(local.updatedAt) ? remote : local;`
	`}`

3	**Langkah 3. Tambahkan aksi sync() ke kelas NoteActions di note_providers.dart.**
	`import '../data/sync.dart';`
	`import 'offline_providers.dart';`
	
	`Future<int> sync() async {`
	`  final offline = _ref.read(forceOfflineProvider);`
	`  final count = await syncNotes(_repo, offline: offline);`
	`  _refresh();`
	`  return count;`
	`}`
	`}`

4	**Langkah 4. Buat model lib/data/remote/post.dart**

week5_offline_notes > lib > data > remote > post.dart > ...

```dart
class Post {
  const Post({required this.id, required this.title, required this.body});
  final int id;
  final String title;
  final String body;
  factory Post.fromJson(Map<String, dynamic> json) => Post(
    id: (json['id'] as num).toInt(),
    title: json['title'] as String? ?? '',
    body: json['body'] as String? ?? '',
  );
  Map<String, dynamic> toJson() => {'id': id, 'title': title, 'body': body};
}
```

5 **Buat repository posts dengan cache lib/data/repositories/post_repository.dart**

week5_offline_notes > lib > data > repositories > post_repository.dart > ...

```dart
import 'dart:convert';

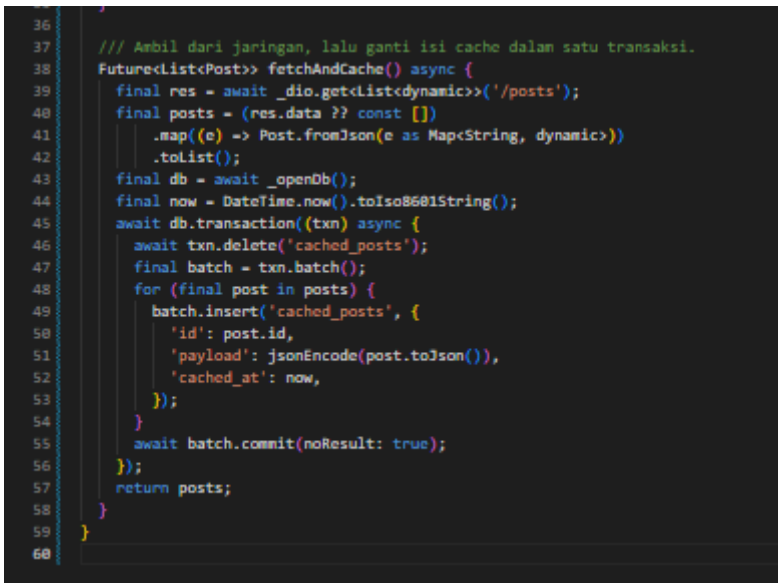
import 'package:dio/dio.dart';
import 'package:sqflite/sqflite.dart';

import '../local/db.dart';
import '../remote/post.dart';

class PostRepository {
  PostRepository({Dio? dio, Future<Database> Function()? openDb})
    : _dio =
          dio ??
          Dio(
            BaseOptions(
              baseUrl: 'https://jsonplaceholder.typicode.com',
              connectTimeout: const Duration(seconds: 5),
              receiveTimeout: const Duration(seconds: 5),
            ), // BaseOptions
          ), // Dio
      _openDb = openDb ?? openNotesDb;
  final Dio _dio;
  final Future<Database> Function() _openDb;

  /// Baca cache dari tabel cached_posts (tanpa jaringan).
  Future<List<Post>> readCachedPosts() async {
    final db = await _openDb();
    final rows = await db.query('cached_posts', orderBy: 'id ASC');
    return rows
        .map(
          (r) => Post.fromJson(
            jsonDecode(r['payload'] as String) as Map<String, dynamic>,
          ),
        )
        .toList();
  }

```



```
  /// Ambil dari jaringan, lalu ganti isi cache dalam satu transaksi.
  Future<List<Post>> fetchAndCache() async {
    final res = await _dio.get<List<dynamic>>('/posts');
    final posts = (res.data ?? const [])
        .map((e) => Post.fromJson(e as Map<String, dynamic>))
        .toList();
    final db = await _openDb();
    final now = DateTime.now().toIso8601String();
    await db.transaction((txn) async {
      await txn.delete('cached_posts');
      final batch = txn.batch();
      for (final post in posts) {
        batch.insert('cached_posts', {
          'id': post.id,
          'payload': jsonEncode(post.toJson()),
          'cached_at': now,
        });
      }
      await batch.commit(noResult: true);
    });
    return posts;
  }
}
```

6	**Langkah 6. Buat provider cache-first lib/providers/post_providers.dart.**

week5_offline_notes > lib > providers > post_providers.dart > PostsNotifier > build

```dart
import 'dart:async';

import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../data/remote/post.dart';
import '../data/repositories/post_repository.dart';
import '../data/sync.dart';
import 'offline_providers.dart';

final postRepositoryProvider = Provider<PostRepository>(
  (ref) => PostRepository(),
); // Provider
final postsProvider = AsyncNotifierProvider<PostsNotifier, List<Post>>(
  PostsNotifier.new,
);

class PostsNotifier extends AsyncNotifier<List<Post>> {
  @override
  Future<List<Post>> build() async {
    final repo = ref.watch(postRepositoryProvider);
    final offline = ref.watch(forceOfflineProvider);
    // 1. Selalu baca cache lokal terlebih dahulu.
    final cached = await repo.readCachedPosts();
    if (offline) {
      if (cached.isEmpty) {
        throw const OfflineException(
          'Offline dan belum ada cache. Buka halaman ini sekali saat online.',
        );
      }
      return cached;
    }
    // 2. Belum ada cache: terpaksa menunggu jaringan.
    if (cached.isEmpty) return repo.fetchAndCache();
    // 3. Ada cache: tampilkan segera, refresh di background.
    unawaited(_refreshInBackground(repo));
    return cached;
  }

  Future<void> _refreshInBackground(PostRepository repo) async {
    try {
      final fresh = await repo.fetchAndCache();
      state = AsyncData(fresh);
    } catch (_) {
      // Jaringan gagal: tetap tampilkan cache, jangan ganti UI dengan error.
    }
  }

  /// Dipanggil oleh pull-to-refresh. Mengembalikan true bila berhasil.
  Future<bool> refresh() async {
    if (ref.read(forceOfflineProvider)) return false;
    try {
      final fresh = await ref.read(postRepositoryProvider).fetchAndCache();
      state = AsyncData(fresh);
      return true;
    } catch (_) {
      return false; // data lama (cache) tetap ditampilkan
    }
  }
}
```

7	**Langkah 7. Buat halaman posts lib/pages/posts_page.dart**

week5_offline_notes > lib > pages > posts_page.dart > PostsPage > build

```dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../providers/offline_providers.dart';
import '../providers/post_providers.dart';

class PostsPage extends ConsumerWidget {
  const PostsPage({super.key});
  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final postsAsync = ref.watch(postsProvider);
    final offline = ref.watch(forceOfflineProvider);
    return Scaffold(
      appBar: AppBar(title: const Text('Posts (cache-first)')),
      body: Column(
        children: [
          if (offline)
            Container(
              width: double.infinity,
              color: Colors.orange.shade100,
              padding: const EdgeInsets.all(12),
              child: const Text(
                'Mode offline: menampilkan data dari cache lokal',
                style: TextStyle(color: Colors.black87),
              ), // Text
            ), // Container
          Expanded(
            child: postsAsync.when(
              loading: () => const Center(child: CircularProgressIndicator()),
              error: (e, _) => Center(
                child: Padding(
                  padding: const EdgeInsets.all(24),
                  child: Text('$e', textAlign: TextAlign.center),
                ), // Padding
              ), // Center
              data: (posts) => RefreshIndicator(
                onRefresh: () async {
                  final ok = await ref.read(postsProvider.notifier).refresh();
                  if (!ok && context.mounted) {
                    ScaffoldMessenger.of(context).showSnackBar(
                      const SnackBar(
                        content: Text('Gagal refresh, menampilkan cache'),
                      ), // SnackBar
                    );
                  }
                },
                child: ListView.builder(
                  itemCount: posts.length,
                  itemBuilder: (context, index) {
                    final post = posts[index];
                    return ListTile(
                      leading: CircleAvatar(child: Text(post.id.toString())),
                      title: Text(post.title, maxLines: 1),
                      subtitle: Text(post.body, maxLines: 2),
                    ); // ListTile
                  },
                ), // ListView.builder
              ), // RefreshIndicator
            ),
          ), // Expanded
        ],
      ), // Column
    ); // Scaffold
  }
}
```

8	**Langkah 8. Tambahkan tombol Posts dan Sinkronkan di AppBar NotesPage**

```
import '../data/sync.dart';
import 'posts_page.dart';
```

```
        ), // Tooltip
        // Sisipkan dua IconButton ini di dalam actions AppBar,
        // sebelum tombol Pengaturan:
        IconButton(
          tooltip: 'Posts (cache-first)',
          icon: const Icon(Icons.article_outlined),
          onPressed: () =>
              Navigator.of(context)
                  .push(MaterialPageRoute(builder: (_) => const PostsPage())),
        ), // IconButton
        IconButton(
          tooltip: 'Sinkronkan',
          icon: const Icon(Icons.sync),
          onPressed: () async {
            final messenger = ScaffoldMessenger.of(context);
            try {
              final count = await ref.read(noteActionsProvider).sync();
              messenger.showSnackBar(
                SnackBar(
                  content: Text(
                    count == 0
                        ? 'Semua catatan sudah tersinkron'
                        : '$count catatan berhasil disinkronkan',
                  ), // Text
                ), // SnackBar
              );
            } on OfflineException catch (e) {
              messenger.showSnackBar(SnackBar(content: Text(e.message)));
            }
          },
        ), // IconButton
```

9	**Langkah 9. Tambahkan saklar offline di SettingsPage**

```
import '../providers/offline_providers.dart';
```

```
    final offline = ref.watch(forceOfflineProvider);
```

```

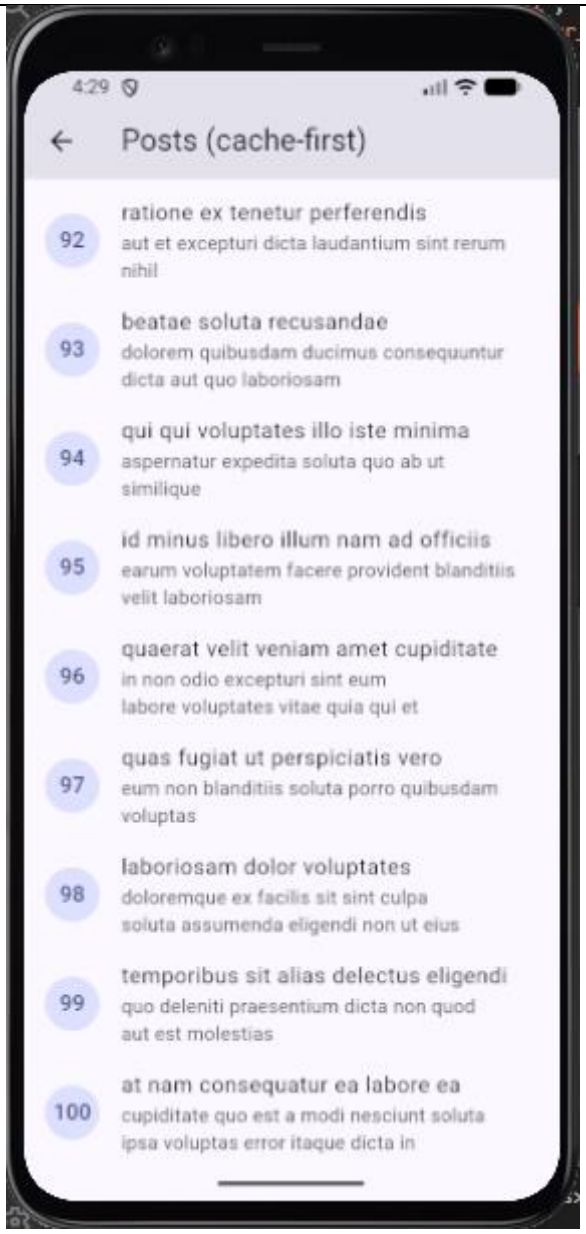
          SwitchListTile(
            secondary: const Icon(Icons.wifi_off),
            title: const Text('Paksa mode offline'),
            subtitle: const Text('Simulasi offline untuk demo dan testing'),
            value: offline,
            onChanged: (_) => ref.read(forceOfflineProvider.notifier).toggle(),
          ), // SwitchListTile
        ],
      ), // ListView
```

10	**Langkah 10. Lakukan uji offline-first dan dokumentasikan.**

No	Skenario	Langkah	Hasil yang diharapkan
1	Isi cache	Online, buka halaman Posts	100 posts tampil; tersimpan di `cached_posts`
2	Cache saat offline	Aktifkan mode pesawat, tutup lalu buka aplikasi, buka Posts	Posts tetap tampil dari cache
3	Antrean dirty	Masih offline, tambah 3 catatan	Badge menunjukkan angka 3
4	Sync ditolak	Aktifkan **Paksa mode offline**, tekan Sinkronkan	Snackbar "Perangkat offline…", badge tetap 3
5	Sync berhasil	Matikan saklar offline, tekan Sinkronkan	Setelah ±1 detik snackbar "3 catatan…", badge hilang, ikon awan hijau
6	Refresh background	Online, buka Posts	Daftar tampil seketika dari cache, lalu diperbarui

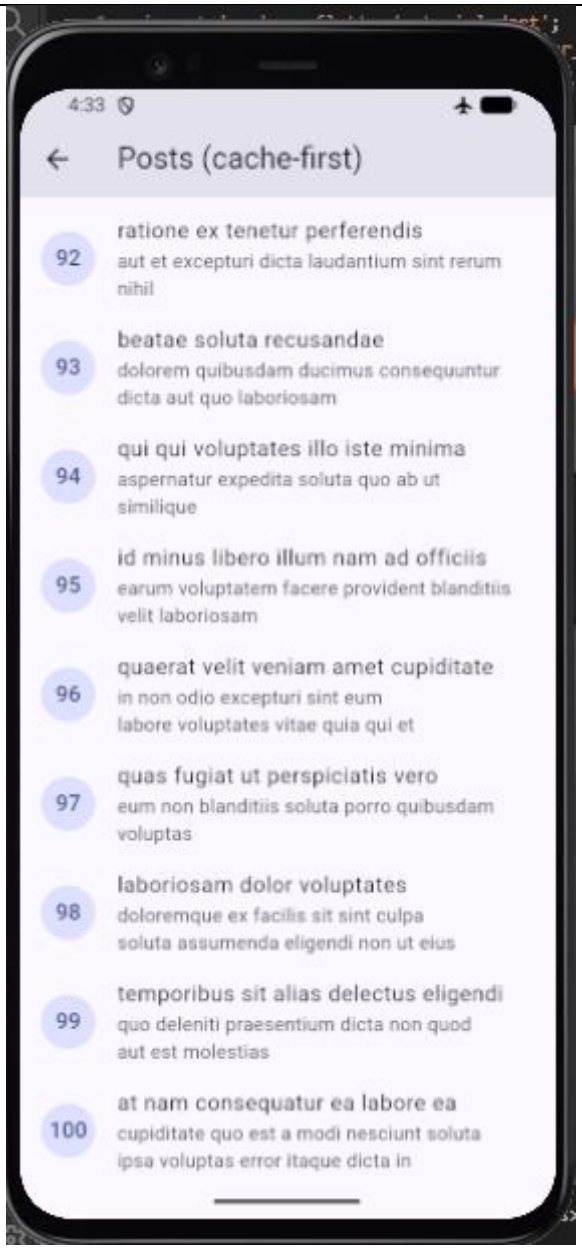
1. Isi cache


4:29
Posts (cache-first)
1
sunt aut facere repellat provident
quia et suscipit
suscipit recusandae consequuntur expedita
2
qui est esse
est rerum tempore vitae
sequi sint nihil reprehenderit dolor beatae ea
3
ea molestias quasi exercitationem
et iusto sed quo iure
voluptatem occaecati omnis eligendi aut ad
4
eum et est occaecati
ullam et saepe reiciendis voluptatem adipisci
sit amet autem assumenda provident rerum
5
nesciunt quas odio
repudiandae veniam quaerat sunt sed
alias aut fugiat sit autem sed est
6
dolorem eum magni eos aperiam quia
ut aspernatur corporis harum nihil quis
provident sequi
7
magnam facilis autem
dolore placeat quibusdam ea quo vitae
magni quis enim qui quis quo nemo aut saepe
8
dolorem dolore est ipsam
dignissimos aperiam dolorem qui eum
facilis quibusdam animi sint suscipit qui sint
9
nesciunt iure omnis dolorem tempora
consectetur animi nesciunt iure dolore
enim quia ad
optio molestias id quia eum

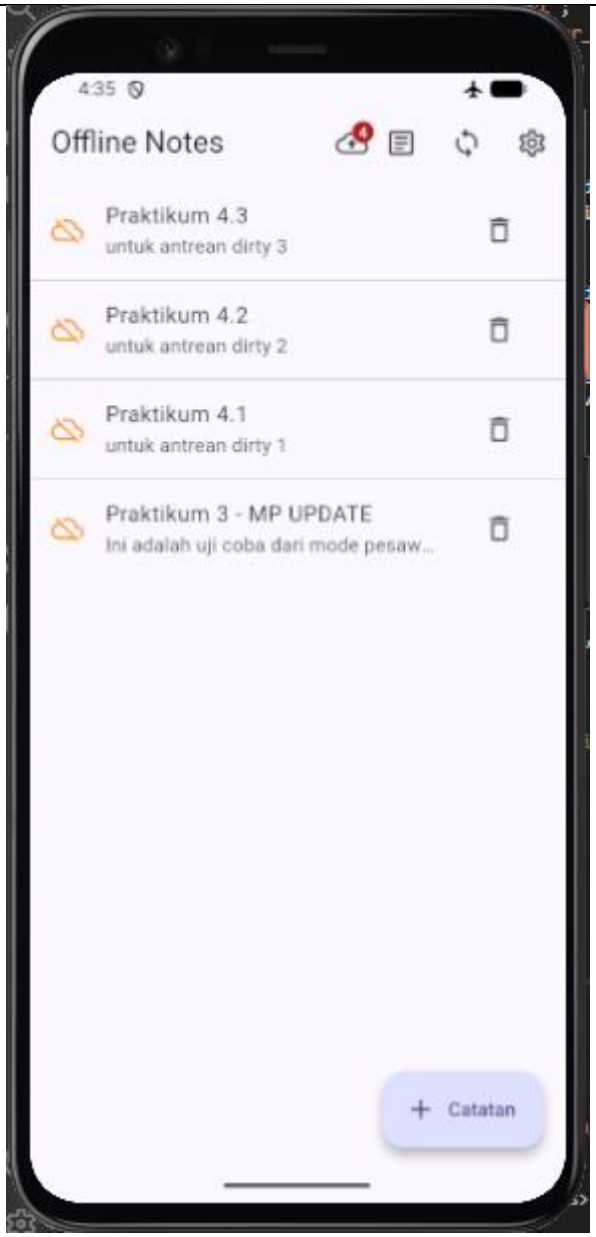


2. Cache saat offline

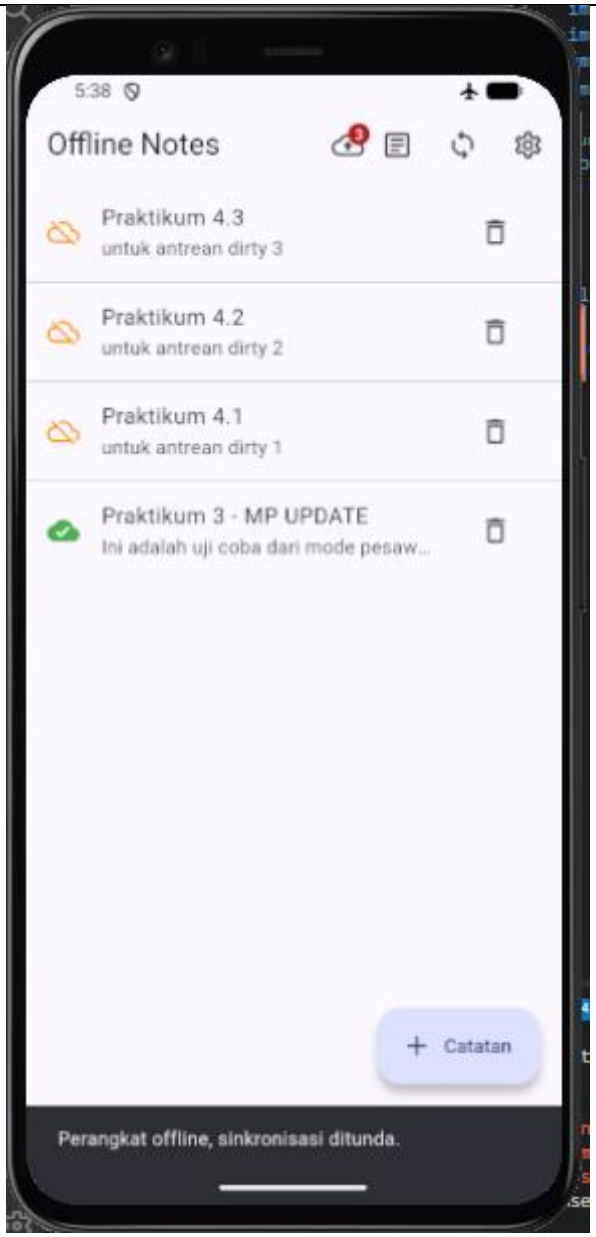

4:33
Posts (cache-first)
1
sunt aut facere repellat provident
quia et suscipit
suscipit recusandae consequuntur expedita
2
qui est esse
est rerum tempore vitae
sequi sint nihil reprehenderit dolor beatae ea
3
ea molestias quasi exercitationem
et iusto sed quo iure
voluptatem occaecati omnis eligendi aut ad
4
eum et est occaecati
ullam et saepe reiciendis voluptatem adipisci
sit amet autem assumenda provident rerum
5
nesciunt quas odio
repudiandae veniam quaerat sunt sed
alias aut fugiat sit autem sed est
6
dolorem eum magni eos aperiam quia
ut aspernatur corporis harum nihil quis
provident sequi
7
magnam facilis autem
dolore placeat quibusdam ea quo vitae
magni quis enim qui quis quo nemo aut saepe
8
dolorem dolore est ipsam
dignissimos aperiam dolorem qui eum
facilis quibusdam animi sint suscipit qui sint
9
nesciunt iure omnis dolorem tempora
consectetur animi nesciunt iure dolore
enim quia ad
optio molestias id quia eum



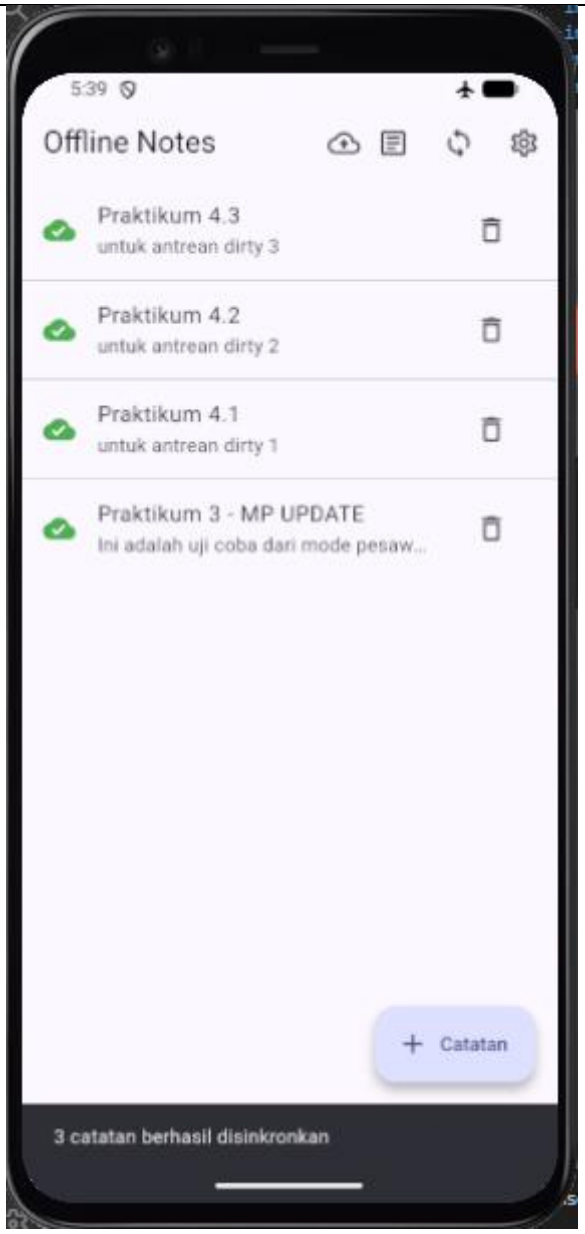
3. Antrean dirty



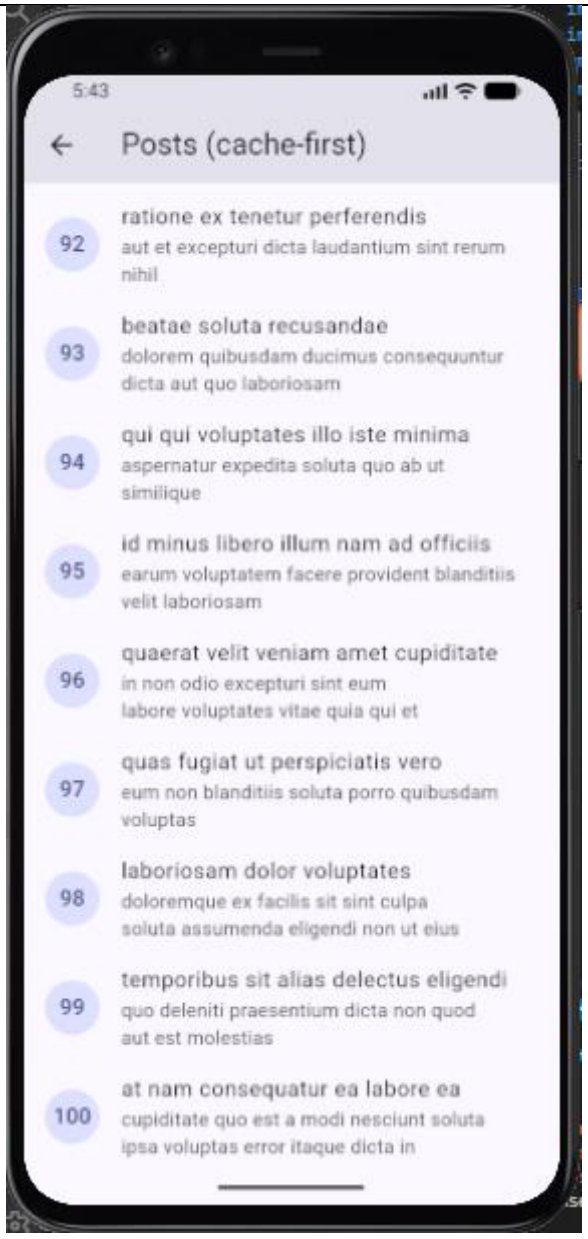
4. **Sync ditolak**



5. Sync berhasil



6. Refresh background



Pertanyaan Praktikum 4

1. Apa perbedaan cache-first dan network-first? Berikan satu contoh data yang lebih cocokmemakai network-first.

   Jawaban:

   - Cache-first menggunakan data yang sudah tersimpan terlebih dahulu agar data bisa langsung ditampilkan, sedangkan network-first mencoba mengambil data terbaru dari internet terlebih dahulu. Contoh data yang lebih cocok menggunakan network-first adalah harga barang, karena datanya harus selalu terbaru.

2. Jelaskan skenario kehilangan data yang dapat terjadi akibat markAllSynced(), lalu usulkanperbaikannya.

   Jawaban:

   - markAllSynced() bisa menyebabkan data dianggap sudah tersinkron padahal sebenarnya belum berhasil dikirim ke server. Perbaikannya, status dirty sebaiknya diubah menjadi 0 hanya setelah server benar-benar menerima data dengan sukses.

3. Mengapa diperlukan saklar forceOffline padahal sudah ada mode pesawat?

   Jawaban:

   - forceOffline diperlukan agar kondisi offline bisa diuji secara sengaja tanpa harus mengaktifkan mode pesawat. Jadi, pengujian fitur offline menjadi lebih mudah dan terkontrol.

4. Mengapa fetchAndCache() menulis cache di dalam transaksi?

   Jawaban:

   - Supaya proses penyimpanan cache berjalan dengan aman. Jika terjadi kesalahan saat proses berlangsung, perubahan bisa dibatalkan sehingga data cache tidak menjadi tidak lengkap atau rusak.

Praktikum 5 — Pengujian dengan Repository Palsu

Langkah	Jawaban/Deskripsi
1	**Langkah 1. Hapus test bawaan dan buat test/note_test.dart** `PS C:\Users\Ryoo\my_first_app\week5_offline_notes> rm test/widget_test.dart` Test bawaan (widget_test.dart) menguji aplikasi counter yang sudah tidak ada, sehingga pasti gagal.

week5_offline_notes > test > note_test.dart > main

```dart
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:week5_offline_notes/data/local/note.dart';
import 'package:week5_offline_notes/data/repositories/note_repository.dart';
import 'package:week5_offline_notes/data/sync.dart';
import 'package:week5_offline_notes/providers/note_providers.dart';

class FakeNoteRepository extends NoteRepository {
  FakeNoteRepository({List<Note> items = const [], this.throwError = false})
    : items = [...items],
      super(openDb: () => throw UnimplementedError());
  final List<Note> items;
  final bool throwError;
  @override
  Future<List<Note>> fetchNotes() async {
    if (throwError) throw Exception('db locked (simulasi)');
    return items;
  }

  @override
  Future<int> countDirty() async => items.where((n) => n.dirty).length;
  @override
  Future<void> markAllSynced() async {
    for (var i = 0; i < items.length; i++) {
      items[i] = items[i].copyWith(dirty: false);
    }
  }
}

Note _note(String title, {bool dirty = false, DateTime? at}) =>
    Note(title: title, updatedAt: at ?? DateTime(2026, 9, 18), dirty: dirty);
Run | Debug
void main() {
  Run | Debug
  group('Model Note', () {
    Run | Debug
    test('fromMap aman terhadap field yang hilang', () {
      final note = Note.fromMap({'title': 'Belanja'});
      expect(note.title, 'Belanja');
      expect(note.body, '');
      expect(note.dirty, isFalse);
    }); // test('fromMap aman terhadap field yang hilang')
    Run | Debug
```

```
Run | Debug
40    test('flag dirty bertahan pada serialisasi', () {
41      final note = _note('a', dirty: true);
42      final restored = Note.fromMap(note.toMap());
43      expect(restored.dirty, isTrue);
44      expect(restored.updatedAt, note.updatedAt);
45    }); // test('flag dirty bertahan pada serialisasi')
46  }); // group('Model Note')
    Run | Debug
47  group('Provider dengan repository palsu', () {
      Run | Debug
48    test('notesProvider sukses', () async {
49      final container = ProviderContainer(
50        overrides: [
51          noteRepositoryProvider.overrideWithValue(
52            FakeNoteRepository(items: [_note('Tes')]),
53          ),
54        ],
55      ); // ProviderContainer
56      addTearDown(container.dispose);
57      final notes = await container.read(notesProvider.future);
58      expect(notes.length, 1);
59      expect(notes.first.title, 'Tes');
60    }); // test('notesProvider sukses')
      Run | Debug
61    test('notesProvider error', () async {
62      final container = ProviderContainer(
63        overrides: [
64          noteRepositoryProvider.overrideWithValue(
65            FakeNoteRepository(throwError: true),
66          ),
67        ],
68      ); // ProviderContainer
69      addTearDown(container.dispose);
70      await expectLater(
71        container.read(notesProvider.future),
72        throwsA(isA<Exception>()),
73      );
74    }); // test('notesProvider error')
75  }); // group('Provider dengan repository palsu')
    Run | Debug
```

```dart
Run | Debug
group('Offline-first', () {
  Run | Debug
  test('syncNotes membersihkan semua catatan dirty', () async {
    final repo = FakeNoteRepository(
      items: [_note('a', dirty: true), _note('b', dirty: true), _note('c')],
    ); // FakeNoteRepository
    final synced = await syncNotes(repo, latency: Duration.zero);
    expect(synced, 2);
    expect(await repo.countDirty(), 0);
  }); // test('syncNotes membersihkan semua catatan dirty')
  Run | Debug
  test('syncNotes ditolak saat offline dan dirty tetap utuh', () async {
    final repo = FakeNoteRepository(items: [_note('a', dirty: true)]);
    await expectLater(
      syncNotes(repo, offline: true),
      throwsA(isA<OfflineException>()),
    );
    expect(await repo.countDirty(), 1);
  }); // test('syncNotes ditolak saat offline dan dirty tetap utuh')
  Run | Debug
  test('resolveConflict memilih updatedAt terbaru', () {
    final local = _note('lokal', at: DateTime(2026, 9, 18, 10));
    final remote = _note('server', at: DateTime(2026, 9, 18, 11));
    expect(resolveConflict(local, remote).title, 'server');
    expect(resolveConflict(remote, local).title, 'server');
  }); // test('resolveConflict memilih updatedAt terbaru')
}); // group('Offline-first')
}
```

2 **Langkah 2. Jalankan analisis statis dan test.**

```
PS C:\Users\Ryoo\my_first_app\week5_offline_notes> flutter analyze
Analyzing week5_offline_notes...
No issues found! (ran in 18.1s)
PS C:\Users\Ryoo\my_first_app\week5_offline_notes> flutter test
01:00 +3 -1: Provider dengan repository palsu notesProvider error [E]
  TimeoutException after 0:00:30.000000: Test timed out after 30 seconds. See https://pub.dev/packages/test#timeouts
  dart:isolate  _RawReceivePort._handleMessage

  Expected: throws <Instance of 'Exception'>
    Actual: <Instance of 'Future<List<Note>>'>
     Which: threw StateError:<Bad state: The provider FutureProvider<List<Note>>#816b1 was disposed during loading state, y
et no value could be emitted.>
            stack package:riverpod/src/core/element.dart 341:70             ElementWithFuture.dispose
                  package:riverpod/src/core/provider_container.dart 1317:15  ProviderContainer._dispose
                  package:riverpod/src/core/provider_container.dart 1335:21  ProviderContainer.dispose
                  ===== asynchronous gap ===========================
                  dart:async                                                 _Completer.completeError
                  package:riverpod/src/core/element.dart 341:19              ElementWithFuture.dispose
                  package:riverpod/src/core/provider_container.dart 1317:15  ProviderContainer._dispose
                  package:riverpod/src/core/provider_container.dart 1335:21  ProviderContainer.dispose

            which is not an instance of 'Exception'

  package:matcher                                  expectLater
  package:flutter_test/src/widget_tester.dart 507:8  expectLater
  test\note_test.dart 70:13                        main.<fn>.<fn>


To run this test again: C:\flutter\bin\cache\dart-sdk\bin\dart.exe test C:/Users/Ryoo/my_first_app/week5_offline_notes/test
/note_test.dart -p vm --plain-name "Provider dengan repository palsu notesProvider error"
01:01 +6 -1: Some tests failed.
PS C:\Users\Ryoo\my_first_app\week5_offline_notes>
```

	Test gagal karena ProviderContainer ter-dispose saat FutureProvider masih dalam proses loading. Kondisi tersebut menimbulkan StateError dan membuat test timeout. Jadi, error terjadi pada proses pengujian/lifecycle Riverpod, bukan pada logika aplikasi.

Pertanyaan Praktikum 5

1. Mengapa kita menguji provider dengan ProviderContainer + overrideWithValue danbukan dengan membuka database asli?

   Jawaban:

   - Agar pengujian tidak bergantung pada database asli. Dengan repository palsu, test menjadi lebih cepat, sederhana, dan mudah dikontrol.

2. Apa manfaat parameter latency pada syncNotes bagi pengujian?

   Jawaban:

   - Untuk mengatur waktu simulasi proses sinkronisasi. Saat testing, kita bisa menggunakan Duration.zero agar test berjalan cepat tanpa harus menunggu 1 detik.

3. Tuliskan satu test tambahan yang menurut Anda penting namun belum ada, besertaalasannya

   Jawaban:

   - Test tambahan yang penting adalah menghapus catatan. Test ini memastikan catatan benar-benar terhapus dari repository dan jumlah dirty ikut diperbarui dengan benar. Ini penting karena fitur hapus merupakan salah satu fungsi utama aplikasi.

AI Prompt Challenge dan Refactoring

1. Prompt yang Digunakan

Aplikasi Flutter Offline Notes: CRUD catatan + preferensi tema. Bandingkan SharedPreferences, Hive, sqflite (SQLite), dan Drift untuk dua kebutuhan ini.

Requirements:

- Kriteria: kompleksitas query, kebutuhan relasi, reaktivitas (stream), type-safety, ukuran boilerplate, dan kemudahan testing.
- Beri rekomendasi final: mana untuk preferensi, mana untuk catatan, beserta alasannya dalam 1 tabel.
- Tunjukkan skema tabel/kotak untuk 1000+ catatan.

- Jelaskan trade-off setiap pilihan.

2. Ringkasan Output AI

AI membandingkan SharedPreferences, Hive, sqflite (SQLite), dan Drift berdasarkan kompleksitas query, relasi, reaktivitas, type-safety, boilerplate, dan kemudahan testing.

Hasilnya menunjukkan bahwa SharedPreferences paling cocok untuk data sederhana seperti preferensi tema. Untuk catatan dalam jumlah banyak, sqflite dan Drift lebih sesuai karena mendukung database, query, dan pengelolaan data yang lebih terstruktur.

Hive juga dapat digunakan untuk menyimpan banyak data, tetapi pada project ini tidak dipilih karena kebutuhan catatan lebih membutuhkan kemampuan query dan struktur database yang fleksibel. Drift memiliki kelebihan pada type-safety dan reactive stream, tetapi dianggap lebih kompleks untuk kebutuhan praktikum ini.

3. Tabel Perbandingan Final

Kriteria	SharedPreferences	Hive	sqflite (SQLite)	Drift
Kompleksitas query	Rendah	Rendah–Sedang	Tinggi	Tinggi
Dukungan relasi	Tidak cocok	Terbatas	Baik	Baik
Reaktivitas (stream)	Tidak utama	Ada	Tidak bawaan	Sangat baik
Type-safety	Rendah	Sedang	Rendah	Tinggi
Ukuran boilerplate	Sangat kecil	Kecil	Sedang	Sedang–Tinggi
Kemudahan testing	Mudah	Mudah–Sedang	Sedang	Baik
Cocok untuk preferensi?	Sangat cocok	Cocok	Kurang cocok	Berlebihan
Cocok untuk 1000+ catatan?	Tidak cocok	Cocok	Sangat cocok	Sangat cocok
Keputusan	Dipilih	Tidak dipilih	Dipilih	Tidak dipilih

4. Verifikasi Hasil AI

- SharedPreferences untuk daftar catatan: Tidak direkomendasikan karena kurang cocok untuk menyimpan koleksi data dalam jumlah banyak.
- Antrean sinkronisasi: Catatan menggunakan dirty dan updated_at sehingga data yang belum tersinkron dapat diketahui.

- Klaim real-time: Kemampuan stream lebih jelas pada Drift karena menyediakan mekanisme watch, sehingga klaim real-time tidak boleh hanya berdasarkan asumsi.
- Boilerplate: SharedPreferences memiliki struktur yang paling sederhana, sedangkan SQLite dan Drift membutuhkan struktur database yang lebih banyak.

5. Keputusan Akhir

Saya memilih SharedPreferences untuk preferensi tema karena data yang disimpan sederhana dan tidak membutuhkan query yang kompleks.

Untuk catatan, saya memilih SQLite/sqflite karena mampu menyimpan banyak data, mendukung query, dan sesuai dengan kebutuhan aplikasi Offline Notes seperti dirty dan updated_at untuk proses sinkronisasi.

week5_offline_notes > docs > perbandingan-storage.md > # Perbandingan Storage Offline Notes

```
# Perbandingan Storage Offline Notes

| Kriteria | SharedPreferences | Hive | sqflite (SQLite) | Drift |
|---|---|---|---|---|
| Kompleksitas query | Rendah | Rendah–Sedang | Tinggi | Tinggi |
| Dukungan relasi | Tidak cocok | Terbatas | Baik | Baik |
| Reaktivitas (stream) | Tidak utama | Ada | Tidak bawaan | Sangat baik |
| Type-safety | Rendah | Sedang | Rendah | Tinggi |
| Ukuran boilerplate | Sangat kecil | Kecil | Sedang | Sedang–Tinggi |
| Kemudahan testing | Mudah | Mudah–Sedang | Sedang | Baik |
| Cocok untuk preferensi? | Sangat cocok | Cocok | Kurang cocok | Berlebihan |
| Cocok untuk 1000+ catatan? | Tidak cocok | Cocok | Sangat cocok | Sangat cocok |
| Keputusan | Dipilih | Tidak dipilih | Dipilih | Tidak dipilih |

Tambahan pada baris keputusan:
SharedPreferences: Dipilih karena sederhana untuk data kecil seperti tema
Hive: Tidak dipilih karena kebutuhan catatan lebih membutuhkan query dan struktur database yang fleksibel
sqflite: Dipilih karena mendukung SQLite, query, dan data banyak
Drift: Tidak dipilih karena lebih kompleks


AI Verification Checklist
- Apakah AI menempatkan daftar catatan di SharedPreferences? (Tolak: rapuh untuk koleksi.)
Jawaban:
AI tidak merekomendasikan SharedPreferences untuk menyimpan daftar catatan karena kurang cocok untuk koleksi data yang banyak.

- Apakah skema AI mendukung antrean sync (dirty flag / updated_at) atau hanya CRUD
polos?
Jawaban:
Untuk catatan digunakan dirty dan updated_at sehingga data yang belum tersinkron dapat diketahui.

- Apakah klaim "real-time" AI didukung stream (Drift watch) atau hanya asumsi?
Jawaban:
Kemampuan stream lebih jelas pada Drift karena menyediakan mekanisme watch. Jadi klaim real-time tidak boleh hanya berdasarkan asumsi.

- Apakah estimasi boilerplate AI masuk akal setelah Anda mencoba instalasinya (flutter pub
add + migrasi skema)?
Jawaban:
SharedPreferences memiliki boilerplate paling sedikit, sedangkan SQLite dan Drift membutuhkan struktur database yang lebih banyak.

- Keputusan final Anda beserta alasannya – boleh berbeda dari rekomendasi AI selama
berargumen.
Jawaban:
Saya memilih SharedPreferences untuk preferensi tema dan SQLite/sqflite untuk catatan karena preferensi hanya berupa data sederhana, sedangkan catatan membutuhkan penyimpanan
banyak data, query, serta penanda sinkronisasi.
```

Pertanyaan Refleksi

1. Mengapa daftar catatan tidak boleh disimpan di SharedPreferences? Apa yang rusak jika aturan ini dilanggar?
   Jawaban:

- SharedPreferences lebih cocok digunakan untuk menyimpan data sederhana, seperti pengaturan tema atau nilai kecil lainnya. Kalau digunakan untuk menyimpan banyak catatan, data akan sulit dikelola karena tidak dirancang untuk query, pencarian, pengurutan, dan perubahan data dalam jumlah banyak. Akibatnya, aplikasi bisa menjadi kurang efisien dan pengelolaan catatan menjadi lebih rumit.

2. Kapan cache-first cukup, dan kapan Anda membutuhkan strategi lain (misalnya network-first untuk data harga real-time)?

   Jawaban:

   - Cache-first sudah cukup jika data yang ditampilkan tidak harus selalu merupakan data terbaru dari server, seperti postingan yang masih bisa dibaca ketika offline. Namun, untuk data yang sering berubah dan harus terbaru, seperti harga barang, lebih baik menggunakan network-first agar aplikasi mengambil data terbaru dari server terlebih dahulu.

3. Bagaimana dirty flag berubah menjadi antrean sync tanpa memblokir UI? Kapan antrean terpisah (tabel outbox) menjadi perlu?

   Jawaban:

   - Dirty flag digunakan untuk menandai catatan yang belum tersinkron. Saat catatan dibuat atau diubah, nilai dirty menjadi true. Ketika proses sinkronisasi dijalankan, aplikasi mencari catatan yang dirty, lalu menyinkronkannya dan mengubah dirty menjadi false. Proses ini dilakukan secara asynchronous sehingga tidak membuat tampilan aplikasi berhenti atau terasa macet. Antrean sync terpisah diperlukan jika aplikasi memiliki banyak jenis operasi yang harus disinkronkan atau setiap perubahan perlu dicatat satu per satu agar tidak ada proses yang terlewat.

4. Bagian mana dari rekomendasi AI yang Anda tolak, dan mengapa?

   Jawaban:

   - Saya tidak memilih Hive untuk penyimpanan catatan meskipun Hive sebenarnya bisa menyimpan banyak data. Saya lebih memilih SQLite/sqflite karena lebih sesuai dengan kebutuhan aplikasi saya yang membutuhkan query dan struktur database yang jelas.

Kesimpulan:

Hal terpenting yang saya pelajari dari proyek ini adalah bagaimana membuat aplikasi Flutter yang tetap dapat digunakan meskipun sedang offline. Saya belajar menggunakan SharedPreferences untuk menyimpan pengaturan sederhana, SQLite untuk menyimpan data catatan secara permanen, serta menerapkan dirty flag untuk menandai data yang belum tersinkron. Selain itu, saya juga memahami bahwa penggunaan repository dan Riverpod membuat pengelolaan data menjadi lebih teratur. Dari proyek ini, saya jadi lebih memahami konsep offline-first dan bagaimana data lokal dapat digunakan terlebih dahulu sebelum melakukan sinkronisasi.